Swing Detection - History, Analysis and Architecture

Documents the iterative tuning and architectural fixes for swing detection across all three tools: sensordata*plotter.py, emg*util*yaml*runner.py, emg_util.py.

Table of Contents

1. What Is a Swing?
2. Detector Iteration History
3. Cross-Tool Discrepancy - Root Cause Analysis
4. Fix 1 - HysteresisSwingDetector Bugs
5. Fix 2 - emg_util Z-Score Replaced with Fixed Threshold
6. Fix 3 - Time Axis Unification (abs_us)
7. Detector Reference
8. Future Improvements
9. Appendix - Why EMG Cannot Gate the IMU Baseline

What Is a Swing?

A tennis stroke has two to three distinct IMU events, not one:

1. Back-swing - racket moves backward; generates a gyro spike (~300-500 dps)
2. Forward swing / contact - the actual strike; generates a larger spike (~600-1700 dps)
3. (sometimes) Follow-through - deceleration; may generate a third smaller spike

A naive gyro-threshold detector counts all three as separate swings. The goal is to count one detection per tennis stroke.

Detector Iteration History

Attempt 1 - gyro_only, thr=150 dps, debounce=1.0s

Result: 26 swings / 60s Problems: 150 dps too low - catches arm repositioning, slow drift, table bumps. debounce=1.0s not enough to merge back-swing + forward swing.

Attempt 2 - gyro*and*acc, thr=150 dps, debounce=1.0s

Result: 25 swings Analysis: Adding acc >= 1.5g barely helped at 150 dps - most false positives already had enough acc from gravity + arm movement.

Attempt 3 - gyro*and*acc, thr=300 dps, debounce=1.0s

Result: 17 swings Analysis: 300 dps targets only real fast strokes. Still double-counting back-swing + forward swing pairs within 1-2s.

Attempt 4 - gyro*and*acc, thr=300 dps, debounce=2.0s [Current Default]

Result: 14 swings / 60s Analysis: 17 to 14: dropped 3 back-swing/forward-swing double-counts. 14 / 60s = one stroke every ~4.3s - realistic rally/drill pace.

Cross-Tool Discrepancy - Root Cause Analysis

Problem (discovered 2026-03)

Three tools processing the same full session (onetest/, 1200s wall-clock, 808s physical sensor time) gave very different swing counts:

Tool	Algorithm	Count
emg*util*yaml_runner.py	Adaptive z-score (z>5, baseline=first 2s)	972 raw -> 113 after gap filter -> 92 after EMG filter
sensordata_plotter.py --swing-detector threshold	Fixed 300 dps + acc>=1.5g, debounce=2s	216
sensordata_plotter.py --swing-detector hysteresis	Hysteresis th_on=350, debounce ignored (bug)	351

Root Causes

emg*util*yaml_runner.py (972 -> 113 -> 92) - three compounding problems:

1. Z-score baseline window corrupted - baseline = first 2s of session. If the player was already moving at capture start, the baseline median was inflated, making the z-score threshold too low. Diagnostic showed the baseline window had gyro median ~120 dps (player already active) instead of the true rest floor of ~20 dps.

2. min*gap*ms=300 instead of 2000 - only 300ms between swings allowed, vs the plotter 2s debounce. This caused every back-swing + forward swing pair to be counted as two separate swings.

3. Z-score cannot produce a 300 dps threshold from rest statistics - diagnostic analysis showed the rest floor is ~20 dps with MAD ~6 dps. rest_median + 5 * 1.4826 * MAD = 20 + 44 = 64 dps - far below 300 dps. The gap between general movement (~100-200 dps) and real swings (~300+ dps) cannot be bridged by rest floor statistics alone for wrist-mounted sensors. The distribution is a continuous ramp, not bimodal.

HysteresisSwingDetector (351 swings) - two bugs:

1. --debounce-sec CLI argument was completely ignored - HysteresisSwingDetector had no debounce_sec parameter at all.
2. th_on=350 was higher than ThresholdSwingDetector 300 dps, misaligning the two detectors.

Time axis mismatch (runner 169 vs plotter 216 after all other fixes):

- Plotter used host*rx*ms (wall-clock, 1200s span including BLE transmission delay)
- Runner used dev_us / 1000 (device uptime, 808s span - true physical time)
- Same 2s debounce on different time axes = different effective physical gap
- host*rx*ms inflates apparent session duration by ~49% for this session

---

Fix 1 - HysteresisSwingDetector Bugs

Changes

Added debounce_sec parameter:

- New **init** parameter: debounce_sec=0.0 (default 0 = backward compatible)
- Added *last*trigger_t = -999.0 to reset()

- Confirmation condition: duration >= min*duration AND (t*peak - *last*trigger*t)* >= *debounce*sec
- Both instantiation sites (live + offline) now pass debounce*sec=float(args.debounce*sec)

Aligned defaults with ThresholdSwingDetector:

- th_on: 350 -> 300 dps
- th_off: 175 -> 150 dps (maintains 2:1 hysteresis ratio)

Quantitative Impact

th*on=350, th*off=175, no debounce (original bug) -> 351 swings th*on=350, th*off=175, debounce=2s (fix 1 only) -> 197 swings th*on=300, th*off=150, debounce=2s (fix 1+2) -> 214 swings thr=300, acc>=1.5g, debounce=2s (threshold) -> 216 swings

216 vs 214 - only 2 apart. Residual gap is irreducible: hysteresis exit at 150 dps sets *last*trigger_t slightly later than threshold detector debounce start, occasionally blocking a nearby swing that TD allows.

Fix 2 - emg_util Z-Score Replaced with Fixed Threshold

Why Adaptive Threshold Cannot Work for Wrist Sensors

Extensive diagnostic analysis on the onetest session (1200s, 192k IMU samples):

Full session gyro distribution: p10 = 37 dps (quiet rest floor) p50 = 120 dps (median - player moving most of the time) p85 = 267 dps (general movement ceiling) p90 = 323 dps (swing population begins) p99 = 978 dps max = 1730 dps

The distribution is a continuous ramp, not bimodal. There is no natural valley between general movement and swings that statistics can find.

Stationary window analysis (1s windows, sorted by gyro std): std<15 dps (strict rest, 46s total): threshold = 87 dps -> 380 swings FAIL std<20 dps (91s total): threshold = 157 dps -> 321 swings FAIL std<25 dps (150s total): threshold = 211 dps -> 271 swings FAIL fixed 300 dps (reference): threshold = 300 dps -> 216 swings PASS

EMG-gated analysis (using EMG envelope to find muscle-quiet windows):

- Even during IMU-stationary windows, EMG median = 267 counts (player gripping racket)

- EMG cannot gate the baseline - forearm muscles are active even at rest during play
- EMG-gated thresholds: 103-141 dps -> 340-360 swings (still too many)

Conclusion: The rest floor (~20 dps) and the swing threshold (~300 dps) are 10x apart. No reasonable statistical multiplier bridges this gap from rest statistics alone. The 300 dps fixed threshold is not a magic number - it sits at p88 of the session distribution, exactly where the swing population begins.

Changes

emg*util*yaml*runner.py* - *detect*swings*from*imu():

- Replaced z-score with fixed thr*gyro*dps threshold
- Data already in dps after load*session*stitched - no LSB conversion needed in runner
- Changed min*gap*ms default: 300 -> 2000 (matches plotter debounce=2s)
- Optional acc gate via thr*acc*g config key (default 0 = disabled)

emg*util.py* - *detect*swing*windows*from_imu():

- Same replacement: z-score -> fixed 300 dps threshold
- Added already*in*dps parameter (default True):
- already*in*dps=True (default): data already in dps/g — used by yaml*runner which calls load*session_stitched() that applies FSR conversion before returning
- already*in*dps=False: raw LSB path — reads gyro*fsr*dps / acc*fsr*g columns from the DataFrame and converts LSB -> dps internally
- detect*swing*windows*from*imu_arrays() wrapper added for cross-platform parity:
- Accepts numpy arrays (same signature as Android emg*util*nopandas version)
- Takes explicit gyro*fsr*dps / acc*fsr*g parameters, injects them as DataFrame columns
- Defaults to already*in*dps=False (raw LSB path) to match nopandas interface
- Legacy parameters (baseline*s, z*thr, min_consec) kept in signature for backward compatibility but ignored

config.yaml:

Before: swing*detection: baseline*s: 2.0 z*thr: 5.0 min*consec: 3 min*gap*ms: 300

After: swing*detection: thr*gyro*dps: 300 # fixed gyro threshold in dps thr*acc*g: 0.0 # optional acc gate (0=disabled, 1.5=strict) min*gap_ms: 2000 # matches plotter debounce=2s

Android Java (SessionCaptureActivity.java) - aligned in 2026-03-07:

- GYRO*SWING*THRESHOLD_DPS: ~150 dps -> 300 dps (matches all other paths)
- MARKER*DEBOUNCE*US: 300ms -> 2000ms (matches plotter debounce=2s)
- Previously Android counted ~3x more "swings" than offline analysis

Alignment After Fix (same host*rx*ms time axis)

ThresholdSwingDetector (300 dps, acc>=1.5g, debounce=2s): 216
HysteresisSwingDetector (th*on=300, th*off=150, debounce=2s): 214 (0.9% diff PASS)
emg*util.detect*swing*windows*from*imu (300 dps, gap=2000ms): 222 (2.8% diff PASS)*
*emg*util*yaml*runner.detect*swings*from_imu (300 dps, gap=2000ms): 221 (2.3% diff PASS)

Fix 3 - Time Axis Unification (abs_us)

Problem: Two Different Time Axes

Tool	Time axis	Session span	Effective debounce
sensordata_plotter.py	host*rx*ms (wall-clock)	1200s	2.0s wall-clock
emg*util*yaml_runner.py	dev_us / 1000 (device uptime)	808s	2.0s physical

The 1200s vs 808s difference (49% inflation) comes from BLE transmission delay accumulated over the session - host*rx*ms records when packets arrived at the PC, not when they were captured by the sensor.

The Timestamp Problem in Detail

Every CSV row has two timestamps:

Timestamp	Source	Properties
dev_us	ESP32 micros(), extended to 64-bit by DeviceClockExtender on host	Accurate physical time within a connection. Resets to 0 on ESP32 reboot. DeviceClockExtender has a ~71min residual offset if device ran >71min without rebooting before reconnect.
host*rx*ms	PC wall-clock at BLE packet receive time	Always monotonically increasing. Includes BLE transmission delay (~10-50ms/sample) and reconnection gaps.

Reconnection Cases

Two distinct cases occur when BLE reconnects:

Case 1 - BLE dropout only (ESP32 keeps running):

- dev_us continues counting - it IS the physical timeline
- host*rx*ms has a gap equal to the dropout duration
- Using host*rx*ms inflates the apparent gap

Case 2 - ESP32 hard reset (dev_us resets to 0):

- dev_us restarts from 0 - cannot be used directly across the reconnection
- host*rx*ms gap gives the correct inter-connection duration
- Using dev_us directly would overlap the two connections

Note: BLE reconnection always causes a new session_id file. ESP32 reset always causes BLE reconnection. But BLE reconnection does NOT necessarily mean ESP32 reset - the device may keep running through a BLE dropout.

Solution: Hybrid abs_us Timeline

capture*loader.load*session*stitched() builds a reconstructed physical timeline abs*us using a hybrid approach:

Within each BLE connection: abs*us* = *dev*us - dev_us.min() # accurate physical time, no BLE jitter

Between connections: gap*us* = *(host*rx*ms*first*of*next - host*rx*ms*last*of*prev) * 1000*
*abs*us*next* = *abs*us*prev*end + gap*us* + *(dev*us - dev_us.min())

Using host*rx*ms gap between connections is correct for both cases:

- BLE dropout only: host*rx*ms gap = actual dropout duration
- ESP32 reset: host*rx*ms gap = reboot + reconnect duration
- DeviceClockExtender 71-min residual: irrelevant - dev_us not used across connections

This is simpler and more robust than trying to detect which case occurred from dev_us continuity.

Unit Test Results

Case 1 - BLE dropout only: span=0->95s, gap=5s PASS Case 2 - ESP32 reset: span=0->120s, gap=30s PASS Case 3 - Mixed (dropout then reset): span=0->165s, gap1=5s gap2=25s PASS

Changes

capture_loader.py:

- load*emg*concat() and load*imu*concat(): added host*rx*ms to default usecols
- load*sessions*titched(): replaced two-case dev*us detection with hybrid host*rx_ms-gap approach

emg*util*yaml_runner.py:

- Replaced load*imu*concat() with load*sessions*titched() in swing detection path
- ts*ms is now abs*us / 1000 - correct physical time for debounce
- Fixed FSR defaults: acc 2g->4g, gyro 250->1000 dps (matches firmware)

sensordata_plotter.py:

- Added --log-dir argument for multi-file / multi-connection sessions
- Replaced manual host*rx*ms CSV reading with capture*loader.load*session_stitched()
- Time axis now uses abs*us (physical time) instead of host*rx_ms (wall-clock)
- Falls back to single-file host*rx*ms path if capture_loader not available

Final Alignment (all on abs_us 808s physical timeline)

ThresholdSwingDetector (300 dps, acc>=1.5g, debounce=2s): 177 Runner detect*swings (300 dps, gap=2000ms): 169 (4.5% diff PASS) HysteresisSwingDetector (th*on=300, th_off=150, debounce=2s): 150 (pre-existing - additional gates)

The HD vs TD difference (150 vs 177) is expected and pre-existing: HD has additional gates (th*acc*hp, th*jerk, min*duration) that reject marginal swings that TD accepts. This is HD intended behavior, not a bug.

Detector Reference

ThresholdSwingDetector (Default)

Parameter	Value	Rationale
thr*gyro*dps	300 dps	Sits at p88 of session distribution - swing population begins here
thr*acc*g	1.5 g	Rejects slow drift; at rest ~1.0g due to gravity
debounce_sec	2.0 s	Merges back-swing + forward swing into one count
min*peak*gyro_dps	200 dps	Safety net if threshold is lowered; redundant at thr=300
mode	gyro*and*acc	Both gyro AND acc must exceed thresholds

Logic: above = gmag >= thr*gyro*dps AND amag >= thr*acc*g if above: track peak if peak ends AND peak*gmag* >= *min*peak*gyro*dps AND gap >= debounce_sec: confirm swing

HysteresisSwingDetector (Advanced)

Parameter	Value	Rationale
th_on	300 dps	Aligned with ThresholdSwingDetector entry
th_off	150 dps	2:1 hysteresis ratio - natural swing exit
th*acc*hp	0.6 g	High-pass acc threshold for impact detection
th_jerk	80 g/s	Jerk threshold for impact detection
min_duration	0.08 s	Rejects noise spikes shorter than 80ms
debounce_sec	2.0 s	Same as ThresholdSwingDetector for fair comparison

Advantages over ThresholdSwingDetector:

- Natural state machine exit (gmag < 150) vs crude 2s forced gap
- Impact detection via high-pass accel + jerk
- Better for fast rallies (<2s between strokes)

When to use each:

Use Case	Recommended
Quick stroke counting	threshold (default, simpler)
EMG-impact correlation	hysteresis (provides impact times)
Fatigue analysis	hysteresis (impact = peak muscle activation)

Use Case	Recommended
Fast rallies (<2s between strokes)	hysteresis (no forced debounce gap)

emg*util*yaml*runner / emg*util

Parameter	Config Key	Default	Notes
Gyro threshold	thr*gyro*dps	300 dps	Same as plotter
Acc gate	thr*acc*g	0.0 (disabled)	Set to 1.5 to match ThresholdSwingDetector
Min gap	min*gap*ms	2000 ms	Matches plotter debounce=2s
Window pre/post	pre*ms / post*ms	250 / 450 ms	EMG feature extraction window
Data format	already*in*dps	True (yaml_runner)	yaml*runner: dps (load*session_stitched converts); arrays wrapper: False (raw LSB)

Android Java (SessionCaptureActivity.java)

Parameter	Old Value	Current Value	Notes
GYRO*SWING*THRESHOLD_DPS	~150 dps	300 dps	Aligned with all other paths
MARKER*DEBOUNCE*US	300,000 µs (300ms)	2,000,000 µs (2000ms)	Aligned with plotter debounce=2s

---

Back-Swing vs Forward Swing - Why debounce=2.0s Works

A typical tennis forehand timeline:

t=0.0s - back-swing starts (gyro rises) t=0.3s - back-swing peak (~300-500 dps) <- detector triggers here t=0.8s - transition (gyro drops below threshold) t=1.2s - forward swing starts (gyro rises again) t=1.5s - contact peak (~600-1200 dps) <- detector would trigger again t=1.8s - follow-through (gyro drops)

With debounce=1.0s: gap = 1.2s > 1.0s -> both confirmed (double-count) With debounce=2.0s: gap = 1.2s < 2.0s -> forward swing suppressed

---

Future Improvements

1. Direction-aware detection - use sign of dominant gyro axis to distinguish back-swing (negative rotation) from forward swing (positive rotation). Would allow debounce=0.5s while still counting only forward swings. Requires knowing sensor orientation on wrist.
2. Per-player threshold calibration - 300 dps is correct for an average adult player. Juniors or players with weaker strokes may need 200-250 dps. Could be set via --imu-thr-dps or a per-player config entry.
3. Stroke classifier - ML model trained on gyro+acc window to classify forehand / backhand / serve. Would give stroke type in addition to count.
4. Racket-mounted sensor - for 100% impact detection, mount sensor on racket handle instead of wrist. Wrist sensor achieves ~60% impact detection rate because grip absorption and off-center hits reduce shock transmission.
5. Plotter multi-session breakdown - --log-dir currently loads all connections as one stitched session. Could add per-connection swing rate display to show fatigue progression across reconnections.

Appendix - Why EMG Cannot Gate the IMU Baseline

During the adaptive threshold investigation, EMG was considered as a gate to find true rest windows (IMU stationary AND EMG quiet = no muscle activation).

Result: Even during IMU-stationary windows (gyro std < 15 dps), the EMG envelope median was 267 counts (p75 = 729 counts). The player is constantly gripping the racket, maintaining forearm muscle tone even when standing still.

This means:

- IMU stationary + EMG quiet windows are very rare (<30 windows = 15s total)
- Those windows give a rest floor of ~29 dps -> threshold = 130 dps (still too low)
- EMG cannot reliably separate arm at rest from arm moving but not swinging for a wrist-mounted sensor during a tennis session

Conclusion: EMG is useful for filtering swing windows by electrode quality (already implemented via capture_health.py), but cannot improve the gyro threshold estimation. The fixed 300 dps threshold is the correct solution.

Last Updated: 2026-03-07 (adaptive threshold investigation + time axis unification + Android alignment + already*in*dps parameter)